# PH 3120 – computational Physics Laboratory 1
# CPL 105 – Root Finding

## 01. Bisection Method

I.

(a)

If $f$ is a continuous function on a closed interval $[a,b]$, and $N$ is any number between $f(a)$ and $f(b)$ (inclusive), then there exists at least one number $c$ in the interval $(a,b)$ such that $f(c) = N$.

The function values f(a) and f(b) have opposite signs, then there is at least one value c between a and b where: f(c)=0.

(b)

- If a function f(x) is continuous between two points a and b,
  and f(a) and f(b) have opposite signs,
  then by the Intermediate Value Theorem, there is at least one root between a and b.
  - If f(a) *f(b) > 0 there is no root between a,b points so then we can't use this method.
- The bisection method finds the middle point $c = (a + b)/2$.
- It checks whether the root is between:
  - a and c, or
  - c and b,
    by checking the sign of f(c).
- It repeats the process on the half that contains the sign change.

(c) For having root between δ and γ should be,

y(δ)*y(γ) <0;

And the function must be continuous

(d)

We calculate y(**β** )Then:

- If |y(β)|<tolerance, we assume β is close enough to a root and stop.
- Otherwise, check the sign of y(γ)·y(β):
    - If negative, root lies between γ and β
    - If positive, root lies between β and δ
- Repeat the process with the new interval.

(e)

Considerations:

- Works only if the function is continuous on the interval.
- Requires that the function has opposite signs at the endpoints.

Limitations:

- Slower compared to other methods (like Newton-Raphson).
- Cannot find complex roots.
- Only guarantees convergence if sign change exists.
- Does not use derivative information (less efficient).

II. Done in Jupiter Note book.

III. Done in Jupiter Note book.

# 02. Newton-Raphson Method

I.

(a) Newton-Raphson Method

$$X_{n+1} = X_n - \frac{f(x)}{f'(x)}$$

(b)

The initial guess must be close to the actual root. If it's too far, the method may not work or may take longer to find the root.

(c)

Mathematical Derivation:

From the tangent line at $x_n$ :

$$y - f((x_n\ ) = f\text{'}((x_n\ )(x - (x_n\ )$$

Set y=0 to find where it crosses the x-axis:

$$0 - f(x_n) = f'(x_n)(X_{n+1} - x_n)$$

Solving:

$$X_{n+1} = X_n - \frac{f(x)}{f'(x)}$$

The derivative f′(x) of a function is used in the Newton-Raphson method to determine the slope of the tangent line at the current approximation $X_n$. The next approximation $X_{n+1}$ is found by intersecting this tangent line with the x-axis. This process iteratively refines the root estimate.

(d)

- Derivative is zero or close to zero
  The tangent becomes flat, so the next guess jumps far away or becomes undefined.

- Bad initial guess
  If the guess is far from the root, the method may go in the wrong direction or diverge.

- Function is not smooth or has sharp turns
  The tangent line may give a poor next guess.

- Multiple roots or turning points near the guess
  The method may bounce around or switch roots.

II. Done in Jupiter Note book.

III. Done in Jupiter Note book.

# Python Root Finding Functions

I. Done in Jupiter Note book.

II.

- Only one starting value: fsolve() requires just a single initial guess, while Bisection needs two points with opposite signs.

- Easier to use: fsolve() is built-in and simple to call—no need to write extra code for loops or conditions.
- Faster and smarter: It uses efficient algorithms that often find the root quicker than Bisection.
- Can handle systems: fsolve() can solve multiple equations at once, which Bisection and Newton-Raphson can't do directly.
- No need for derivative: Unlike Newton-Raphson, fsolve() works without needing to calculate the derivative.